blent

Examen du module Python · Formation blent.ai

DigiMarket

Une API REST e-commerce avec Flask

Arnaud Breton

Soutenance : 6 novembre 2026

Sommaire

01
Le projet et son architecture
Le besoin DigiMarket · La factory en 3 couches

02
Les trois couches
Routes · Schemas · Modèles

03
La sécurité
Rôles · Connexion · Token JWT

04
Métier et qualité
Stock et statuts · Tests · Bilan

DigiMarket expose catalogue, comptes et commandes en REST

Utilisateurs

- Inscription : rôle client imposé
- Connexion : token JWT
- Profil, liste et suppression (admin)

Produits

- Lecture publique
- Écriture réservée aux admins
- Pagination et filtres : catégorie, nom, prix

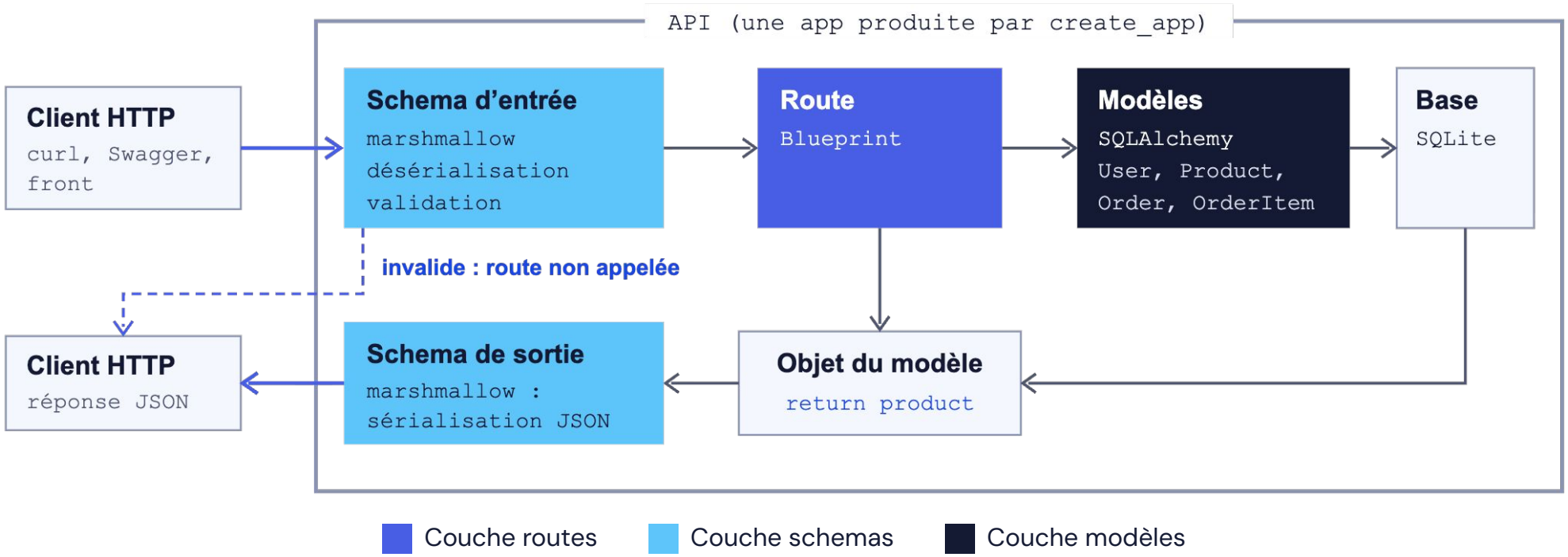
Commandes

- Client : passe et suit ses commandes
- Admin : voit tout, change le statut
- Stock contrôlé, remis à l'annulation

STACK Python 3.13 · APIFlask 3.1 · SQLAlchemy 2.1 · SQLite · JWT · pytest · Ruff

Code : github.com/arnoweb/formation-blentai-module-python Swagger : localhost:5001/docs

Une factory et trois couches : routes, schemas, modèles



Marshalling : convertir les données entre JSON et objets Python, dans les deux sens, avec marshmallow.

L'arborescence du projet : un répertoire par couche

formation-blentai-module-python/	
├─ app.py	point d'entrée : app = create_app()
├─ digimarket/	le package de l'API
│ ├─ __init__.py	create_app() : la factory qui assemble tout
│ ├─ config.py	base SQLite, clé JWT, erreurs de validation en 400
│ ├─ database.py	l'objet db (SQLAlchemy)
│ ├─ auth.py	décorateur admin_required
│ ├─ cli.py	commandes create-admin et set-password
│ ├─ routes/	**couche routes** : health, users, products, orders
│ ├─ schemas/	**couche schemas** : health, user, product, order
│ └─ models/	**couche modèles** : user, product, order
├─ instance/	digimarket.db, la base SQLite
├─ tests/	conftest.py et 5 fichiers test_*.py
└─ requirements.txt, pytest.ini	dépendances et configuration de pytest

Couche routes : un Blueprint par ressource

Blueprint	Préfixe	Accès
users_bp	/api/auth	public, token ou admin selon la route
products_bp	/api/produits	lecture publique, écriture admin
orders_bp	/api/commandes	token, statut réservé admin
health_bp	/health	public

Une route = des décorateurs empilés

```
@products_bp.post("/")
@admin_required
@products_bp.input(ProductCreate)
@products_bp.output(ProductOut,
    status_code=201)
def create_product(json_data):
```

Dans l'ordre : l'URL, la protection, le schéma d'entrée, le schéma de sortie. La fonction reçoit **json_data** déjà validé.

Couche schemas : un contrat d'entrée et de sortie par route

Route (après /api)	Schema d'entrée : ce qui est validé	Schema de sortie : JSON renvoyé
POST /auth/register	RegisterIn : email, password ≥ 8, name	MessageOut, 201
POST /auth/login	LoginIn : email, password	TokenOut, 200
GET /produits/	ProductQuery (query) : page, per_page ≤ 100, filtres	ProductPage, 200
POST /commandes	OrderCreate : adresse, au moins une ligne	OrderDetailOut, 201
PATCH /commandes/{id}	OrderStatusUpdate (query) : statut	OrderOut, 200

Entrée : désérialisation et validation
types, champs requis, bornes ; invalide : 400, la route n'est pas appelée

Sortie : sérialisation
l'objet devient du JSON, seuls les champs déclarés sortent

Couche modèles : quatre tables, chaque ligne de commande fige son prix

Table	Champs clés	Relations
User	email unique, password_hash, rôle	1 → n commandes
Product	nom, catégorie, prix, stock	référencé par les lignes
Order	adresse, statut, date	n lignes, cascade delete-orphan
OrderItem	quantité, prix_unitaire	→ commande, → produit

`prix_unitaire`

copié du produit à la commande : un changement de prix ne touche pas l'historique

`total`

propriété calculée à la volée, jamais stockée : aucune désynchronisation possible

Trois profils, des droits vérifiés à chaque requête

Action	Admin	Client	Visiteur
Lister, supprimer un utilisateur	oui	non (403)	non (401)
Lire le catalogue	oui	oui	oui
Créer, modifier un produit	oui	non (403)	non (401)
Passer une commande	oui	oui	non (401)
Voir des commandes	toutes	les siennes	non (401)
Changer un statut	oui	non (403)	non (401)

Le rôle vient de la base, jamais de la requête : un admin se crée en ligne de commande.

La connexion, fonction par fonction

Entrée : POST /api/auth/login, schéma LoginIn

```
{"email": "client@test.fr", "password": "MotDePasse2026!"}
```

Étape	Fonction ou méthode appelée	Fournie par	Échec
1. Valider le JSON	@users_bp.input(LoginIn)	APIFlask	400
2. Chercher en base	db.select(User).filter_by(email=…)	SQLAlchemy	401
3. Vérifier le hash	check_password_hash(hash, password)	Werkzeug	401
4. Créer le token	create_access_token(identity=str(user.id))	JWT-Extended	aucun

Sortie : schéma TokenOut, 200. Le token porte l'id, pas l'email

```
{"access_token": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiI3In0.…"}
```

Le token JWT : ce qu'il contient, comment il est vérifié

Le token, en 3 parties

Header

```
{"alg": "HS256"}
```

Payload, lisible par tous

```
{"sub": "7", "exp": …}
```

Signature

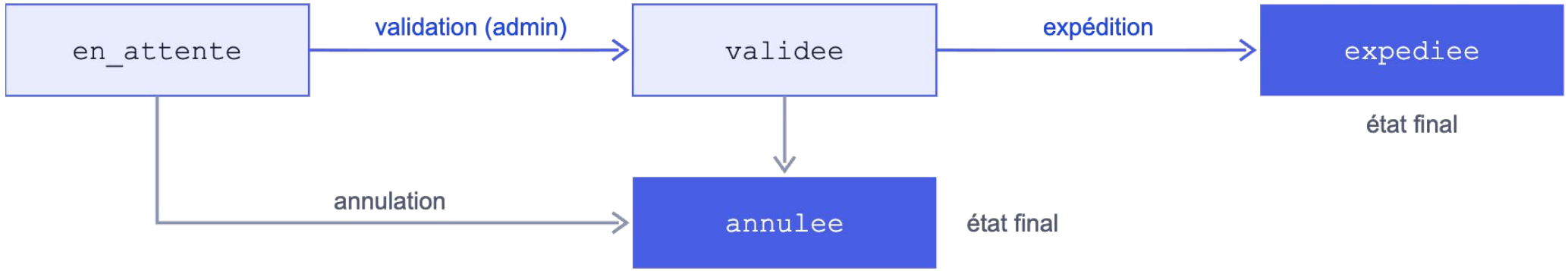
```
HMAC(header.payload, JWT_SECRET_KEY)
```

3 contrôles par requête protégée

Contrôle	Fait par	Échec
1. Token présent ?	Authorization: Bearer	401
2. Signature et date valides ?	@jwt_required()	401
3. Rôle suffisant ?	admin_required, rôle lu en base	403

Les trois passés, la route s'exécute.

Le stock est vérifié deux fois, décrémenté à la validation



Création
stock vérifié, **409** si insuffisant, rien n'est encore réservé

Validation (admin)
stock revérifié puis décrémenté, ligne par ligne

Annulation
après validation, le stock est remis

La qualité : pytest, un hook pre-commit et Ruff

TESTS

pytest

Fixtures dans **conftest.py**. Chaque lancement crée sa propre base SQLite temporaire ; on vérifie le code HTTP et l'état de la base après l'appel.

```
pytest -v
```

GARDE-FOU GIT

Hook pre-commit

Lance pytest avant chaque commit : si un test échoue, le commit est annulé.

```
#!/bin/sh
cd "$(git rev-parse --show-toplevel)" || exit 1
.venv/bin/python -m pytest tests/test_routes.py -v
```

LINTER

Ruff

Vérifie le code lui-même, sans l'exécuter : imports inutilisés, tri des imports, syntaxe moderne.

```
ruff check .  ·  ruff check . --fix
```

Bilan

Ce que ce module m’a apporté, et les évolutions de l’API

Mes acquis

- **Les décorateurs Python** : comprendre comment @input, @output ou @admin_required enveloppent une fonction
- **Flask et APIFlask** : construire une API REST, avec une factory create_app()
- **Les Blueprints** : découper l’API par ressource
- **Les schemas marshmallow** : valider l’entrée, sérialiser la sortie
- **Le contrôle qualité** : tests automatisés avec pytest

Pistes d’évolution de l’API

- Script de seed pour peupler la base
- Une URL plus RESTful : PATCH /api/auth/me plutôt que PUT /api/auth/register/<id>
- PostgreSQL en production, avec Alembic pour versionner les évolutions du schéma SQLAlchemy
- Refresh token et révocation
- Rate limit sur /api/auth/login
- Une passerelle API devant l’application : Apache APISIX
- Intégration continue : tests et Ruff à chaque push

Merci.
Vos questions ?

Le projet sur GitHub

`arnoweb/formation-blentai-module-python`

Me retrouver

Mon GitHub

Mon LinkedIn

Arnaud Breton